

Lab 4 – Vibe Coding

FULL CHAT / LLM HISTORY – 49_brick-breaker

Student	Harshith Kumar
Repository	49_brick-breaker
Remote	BmHarshithKumar/49-brick-breaker
Project	Pygame Brick Breaker
Lab Tasks	1. Collision Detection 2. Game Over 3. Replay/Difficulty 4. Sound Feedback

Transcript note: This document consolidates the available Lab 4 conversation history and the project-work details retained from the conversation. Where the exact original wording was not available, the exchange is represented faithfully as a reconstructed transcript rather than presented as a verbatim quote.

Conversation 1 – Analyze the Broken Brick Breaker

USER

Analyze this Brick Breaker Pygame project, especially **game/ball.py**, **game/brick.py**, **game/paddle.py**, and **game/game_engine.py**. Do not rewrite unrelated code. Identify why the ball currently does not correctly distinguish between top/bottom and left/right collisions with bricks.

ASSISTANT

The collision behavior was caused by the engine treating collisions too generally, so the ball could reverse vertically even when it hit a brick from the side. The correct approach is to use the ball rectangle before movement and after movement. The previous rectangle tells us which side of the target rectangle the ball came from. Top/bottom collisions should change **vy**; left/right collisions should change **vx**. This preserves the existing game structure instead of rewriting unrelated modules.

Task 1 implementation: save **previous_rect = self.ball.rect()** before movement;

self.ball.move(); then calculate **current_rect** and determine the collision side.

Commit: 8bff449 – Fix collision detection

Conversation 2 – Task 1 Side-Based Collision Logic

USER

Give the full code/fix so the ball correctly bounces from the top, bottom, left, and right sides of bricks and the paddle.

ASSISTANT

The engine was updated so paddle and brick collisions use the previous ball position. For a top hit, the previous bottom is at or above the target top and **vy** becomes negative. For a bottom hit, **vy** becomes positive. For a left-side hit, **vx** becomes negative. For a right-side hit, **vx** becomes positive. The brick is then destroyed and the score is incremented as before.

Core logic used:

previous_rect.bottom <= target.top → vy = -abs(vy)

previous_rect.top >= target.bottom → vy = abs(vy)

previous_rect.right <= target.left → vx = -abs(vx)

previous_rect.left >= target.right → vx = abs(vx)

Conversation 3 – Task 2: Game Over

USER

Implement the Game Over condition from the Lab 4 README. It needs an in-game win/loss screen, final score, and input handling.

ASSISTANT

Added game-state variables for the end condition and an end-screen rendering path. The game displays **GAME OVER** when the player loses and **YOU WIN!** when all bricks are cleared. The final score is shown along with an instruction to press a key. Keyboard input is handled while the game is in the end state.

Added: **self.game_over**, **self.result**, end/result fonts, score font, and instruction font.

Important later change: Task 3 replaced the simple exit screen with a replay/difficulty menu.

Conversation 4 – Git Remote and Merge

USER

I need to push the project to my GitHub repository. The remote was initially wrong and the personal repository is **BmHarshithKumar/49-brick-breaker**.

ASSISTANT

The remote was corrected to the personal GitHub repository. Because the remote already contained an initial commit, the histories were unrelated. The repository was pulled with **--allow-unrelated-histories**. A README conflict appeared; the local project README was retained and the merge was committed.

```
git remote set-url origin https://github.com/BmHarshithKumar/49-brick-breaker.git
```

```
git pull --no-rebase origin main --allow-unrelated-histories
```

Merge commit: b4712be – Merge remote repository

Push completed: main → origin/main

Conversation 5 – Task 3 Replay and Difficulty

USER

Implement Task 3 from the README: after the end screen, add a replay option with Easy, Medium, and Hard difficulty, plus an option to exit.

ASSISTANT

The game engine was updated to create a replay/difficulty menu. A reusable **_start_game()** method recreates the paddle, ball, bricks, lives, and score. Difficulty changes both paddle width and ball speed. The existing collision and game-state functionality is preserved.

Easy: paddle width 120, ball speed 3

Medium: paddle width 100, ball speed 4

Hard: paddle width 80, ball speed 6

Controls: 1 = Easy, 2 = Medium, 3 = Hard, Q/Esc = Quit

Commit: 8f83afc – Add replay and difficulty options

Push: b4712be..8f83afc main → main

Conversation 6 – Testing Task 3

USER

I ran **python3 main.py**. Pygame initializes and the terminal returns after the game window is closed. Is that okay?

ASSISTANT

Yes. The terminal returning after closing the Pygame window is normal. The important test is that the game window launches and the implemented gameplay/menu behavior works. Continue with the next lab task after confirming the game itself operates correctly.

Conversation 7 – Task 4 Sound Feedback

USER

Add Sound Feedback according to the README: sound for brick break, paddle/wall hit, and game-over/win.

ASSISTANT

Task 4 was implemented using Pygame's mixer. The game loads four WAV files from **assets/sounds/** and plays them at the corresponding events. Existing Tasks 1–3 remain in the same game engine.

Files created:

assets/sounds/brick.wav – brick break

assets/sounds/hit.wav – paddle/wall hit

assets/sounds/game_over.wav – game over

assets/sounds/win.wav – win

The mixer is initialized with **pygame.mixer.init()**.

Sound calls include **brick_sound.play()**, **hit_sound.play()**, **game_over_sound.play()**, and **win_sound.play()**.

Conversation 8 – Creating the Sound Files

ASSISTANT

The four WAV files were generated locally as simple sound effects so the project could run without relying on external sound assets.

```
python3 - <<'PY'
import wave, math, struct, os
os.makedirs("assets/sounds", exist_ok=True)
# Generate brick.wav, hit.wav, game_over.wav and win.wav
# using short sine-wave effects
PY
```

Conversation 9 – Task 4 Testing

USER

The sound version is working.

ASSISTANT

Good. The next step is to commit the Task 4 source and sound assets, push them, and then verify Git status. Do not commit generated Python cache files or macOS metadata files.

```
git add game/game_engine.py assets/sounds
```

```
git commit -m "Add sound effects"
```

```
git push
```

Conversation 10 – Git Status Cleanup

USER

Git status shows modified **game/__pycache__/*.pyc** files and an untracked **.DS_Store**. It says no changes added to commit.

ASSISTANT

Those are generated/local files and should not be included in the lab submission. Remove them from the working tree and check the status again.

```
rm -rf game/__pycache__
```

```
rm -f .DS_Store
```

```
git status
```

Expected: On branch main; branch up to date with origin/main; nothing to commit, working tree clean.

Conversation 11 – Final Lab Submission

USER

Is the coding part done?

ASSISTANT

Yes. Tasks 1–4 are completed and the project has been pushed to GitHub. The remaining submission items are the final 10-second After Video and the complete Chat/LLM history. The Before Video should document the original broken state, and the After Video should show the completed Tasks 1–4.

Final checklist:

- ✓ Task 1 – Collision detection
- ✓ Task 2 – Game Over
- ✓ Task 3 – Replay + Difficulty
- ✓ Task 4 – Sound feedback
- ✓ GitHub push
- ✓ Exclude __pycache__ and .DS_Store
- Upload final 10-second After Video
- Upload this complete Chat/LLM History PDF

Repository / Commit Record

Stage	Commit / Result
Task 1	8bff449 – Fix collision detection
Remote merge	b4712be – Merge remote repository
Task 3	8f83afc – Add replay and difficulty options
Task 4	Add sound effects – pushed to origin/main

End of consolidated Lab 4 chat history. This PDF is intended for the Lab 4 Chat/LLM-history upload.